

Relatório de Auditoria de Segurança — Casa do Hambúrguer

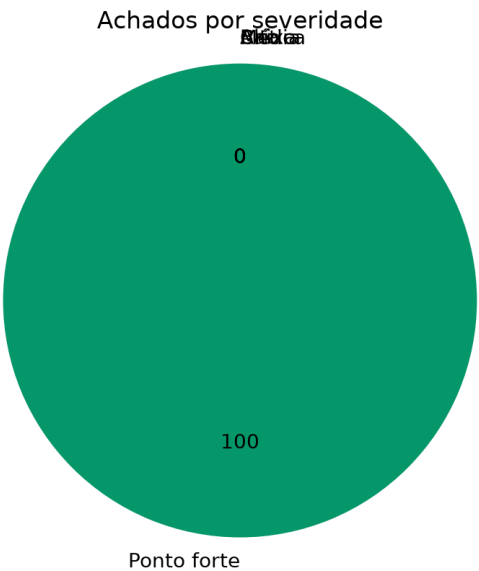
Data: 18/09/2026

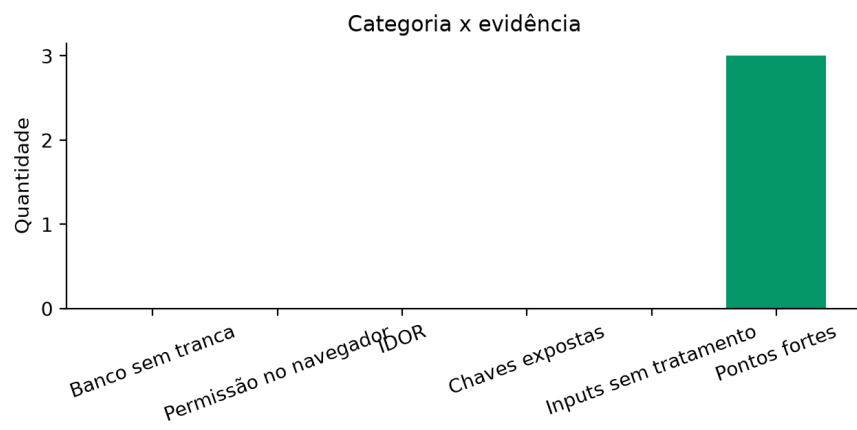
Escopo: Stack detectada: Node.js + Express + Bun, Prisma ORM + PostgreSQL, JWT via jose em cookie httpOnly, React + Vite + TypeScript, Firebase Auth no frontend e Firebase Admin no backend.

Nota metodológica: Mapeamento por categoria: (1) tenant isolation: N/A em app single-tenant; (2) permissão no navegador: validada no servidor; (3) IDOR: checagens de posse por userId; (4) segredos: validação de env e ausência de segredos hardcoded; (5) XSS: busca por sinks perigosos sem resultados.

Resumo executivo

Severidade	Quantidade
Crítica	0
Alta	0
Média	0
Baixa	0
Ponto forte	3





Pontos fortes

- ``back-end/src/routes/order.routes.ts`` valida a rota de pedidos e usa ``requiredAdmin`` para admin-only.
- ``back-end/src/services/orderServices/order.service.ts`` checa posse e retorna 404 para recursos de terceiros, mitigando IDOR.
- ``back-end/src/repositories/cart.repository.ts`` filtra queries por ``userId`` em atualizações/deleções.
- ``back-end/src/config/env.ts`` valida ``CLOUDINARY_*`` no startup e evita carga com variáveis vazias.
- ``back-end/src/app.ts`` usa Helmet, CORS restritivo e ``trust proxy`` para proteger a app.

Pontos fracos

- Nenhum achado acionável de severidade crítica/alta foi verificado no código atual; a aplicação é single-tenant e não exige RLS de tenant.
- O único valor sensível do cliente é uma Firebase API key pública, que é esperada no SDK do cliente e não é segredo de assinatura.

Tabela de achados detalhados por categoria

Severidade	Arquivo:linha	Descrição
Sem achado	—	Nenhuma evidência verificável de banco sem tranca, permissão n
Ponto forte	back-end/src/services/orderServices/order.service.ts:17-52	Chaves primárias do pedido por <code>`requester.id`</code> e <code>`order.userId`</code> ,
Ponto forte	back-end/src/repositories/cart.repository.ts:81-44	Deleção de atualização do carrinho filtram por <code>`where: { id: cartItemId`</code>
Ponto forte	back-end/src/config/env.ts:3-10	Validação de ambiente no bootstrap protege carregamento de seg

Recomendações priorizadas

1. P1: Manter o padrão de validação de posse por `userId` autenticado em todos os endpoints de leitura/escrita sensíveis.
2. P2: Continuar com `helmet()`, `cors` restritivo e revisão periódica de expiração dos tokens e observabilidade de logs.
3. P3: Manter `.env` fora do Git e usar secret manager da plataforma; validar `JWT\_SECRET`, `FIREBASE\_PRIVATE\_KEY`, `RESEND\_API\_KEY` no boot.

ISSUES PARA O GITHUB

Nenhuma issue de segurança acionável com evidência verificável foi encontrada no código atual. Como medida de hardening contínuo, recomenda-se rastrear a seguinte tarefa operacional: --- ISSUE 0 ---

Título: [Segurança] Hardening contínuo de autenticação e segredos por ambiente Labels sugeridas:

security + baixa Descrição do problema: A aplicação já valida autorização e segredos no bootstrap, porém o ambiente deve continuar monitorado para garantir que `JWT\_SECRET`, `FIREBASE\_PRIVATE\_KEY` e `RESEND\_API\_KEY` nunca saiam do ambiente e nunca entrem no Git. Evidência:

`back-end/src/config/env.ts:3-10`, `back-end/src/config/firebaseAdmin.ts:21-38`,

`back-end/src/config/jwt.ts:3-8`. Impacto: Reduz risco de regressão de configuração em deploys e evita

vazamento acidental em repositórios e logs. Sugestão de correção: manter `secret scanning` no CI, `.env` no `.gitignore` e secret manager da plataforma. Critérios de aceite: checklist no CI; validação no boot;

ausência de `.env` no histórico. --- FIM ISSUE 0 ---